МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
**«Российский экономический университет имени Г.В. Плеханова»**

Пермский институт (филиал)

# Отчет

## по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей
(индекс по РУП и наименование профессионального модуля)

Специальность 09.02.07 Информационные системы и программирование

Обучающийся _____ Харин Глеб Алеексеевич
(подпись) (фамилия, имя, отчество)

Группа __ИПс-31____

Руководитель практической подготовки от профильной организации
_____
(должность)

_____ _____
(подпись) (фамилия, имя, отчество)

МП «13» ноября 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«10» ноября 2026 года

Пермь, 2026 год

# Оглавление

ВВЕДЕНИЕ....3
1. ОПИСАНИЕ ЗАДАНИЯ....6
1.1. Назначение модуля импорта товаров....6
1.2. Требования к функциональности и ограничения реализации....6
2. АРХИТЕКТУРА РЕШЕНИЯ....9
2.1. Общая структура проекта ASP.NET MVC....9
2.2. Модель данных и репозиторий в памяти....10
2.3. Сервис импорта и валидация....10
3. ФОРМАТ CSV-ФАЙЛА И РЕАЛИЗАЦИЯ ФУНКЦИОНАЛЬНОСТИ....12
3.1. Описание формата CSV для предметной области «Электроника»....12
3.2. Логика чтения и проверки данных....13
3.3. Разработка интерфейса....15
4. ТЕСТИРОВАНИЕ И ОТЛАДКА....17
4.1. Разработанные тестовые сценарии....17
4.2. Результаты функционального тестирования....18
ЗАКЛЮЧЕНИЕ....22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....25

# ВВЕДЕНИЕ

Актуальность. В современной разработке программного обеспечения важную роль играет интеграция данных из внешних источников. Электронные торговые площадки и корпоративные системы часто используют CSV-файлы для массовой загрузки каталогов товаров, поскольку этот формат является универсальным, легковесным и поддерживается большинством табличных редакторов (Microsoft Excel, Google Таблицы). Ручной ввод сотен наименований товаров с указанием их характеристик (артикул, бренд, цена, гарантия) приводит к ошибкам и требует больших временных затрат. В связи с этим разработка программного модуля, автоматизирующего чтение, валидацию и загрузку данных из CSV-файлов, является актуальной задачей.

Место прохождения практики. Производственная практика проходила в Пермском институте (филиале) федерального государственного бюджетного образовательного учреждения высшего образования «Российский экономический университет имени Г. В. Плеханова» в Приемной комиссии, в период с 03 ноября по 13 ноября 2026 года.

Цель практики: формирование практических умений по разработке требований к программному модулю, его интеграции в веб-приложение, отладке, тестированию и проверке качества кода в рамках профессионального модуля ПМ.02 «Осуществление интеграции программных модулей».

Для достижения поставленной цели в рамках индивидуального задания (Вариант 1 «Электроника») были определены следующие задачи:

1. Проанализировать техническое задание и разработать требования к программному модулю импорта товаров (ПК 2.1).

2. Спроектировать архитектуру веб-приложения на базе ASP.NET MVC с разделением ответственности (модель, сервис импорта, репозиторий, контроллер).

3. Разработать модель данных Product с учетом дополнительных полей для электроники (бренд, срок гарантии, номер модели).

4. Реализовать логику чтения и валидации данных из CSV-файла с сохранением корректных записей в коллекцию в оперативной памяти (без использования баз данных).

5. Выполнить интеграцию разработанного модуля в веб-приложение ASP.NET MVC, создав интерфейс для загрузки файла и просмотра списка товаров (ПК 2.2).

6. Провести отладку программного модуля с использованием инструментов среды разработки Visual Studio (ПК 2.3).

7. Разработать тестовые сценарии и провести функциональное тестирование модуля на предмет обработки некорректных данных (ПК 2.4).

8. Провести инспектирование кода на соответствие базовым стандартам кодирования (ПК 2.5).

9. Оформить отчет по практике и подготовить проект к защите.

Объект исследования: процесс интеграции программных модулей в веб-приложения.

Предмет исследования: разработка модуля импорта данных о товарах из CSV-файлов в ASP.NET_MVC-приложении.

Инструменты разработки: язык программирования C# (.NET 8), фреймворк ASP.NET Core MVC, среда разработки Microsoft Visual Studio 2022, система контроля версий Git.

Структура отчета. Отчет состоит из введения, четырех разделов, заключения, списка использованных источников и приложений. В первом разделе описано задание и требования к модулю. Во втором разделе представлена архитектура решения. В третьем разделе описана реализация

функциональности и работа с CSV. Четвертый раздел посвящен тестированию и отладке программного продукта.

# 1. ОПИСАНИЕ ЗАДАНИЯ

## 1.1. Назначение модуля импорта товаров

Разрабатываемый программный модуль предназначен для автоматизации процесса наполнения каталога электронного магазина товарами. В рамках деятельности сотрудников, отвечающих за контент-менеджмент, часто возникает необходимость массовой загрузки номенклатуры. Ручной ввод данных через веб-интерфейс является медленным и подверженным ошибкам (человеческий фактор).

Модуль решает задачу пакетной обработки данных: он принимает на вход файл формата CSV (Comma-Separated Values), содержащий перечень товаров, и интегрирует его в веб-приложение.

Основное назначение модуля заключается в следующем:

Предоставление пользователю удобного веб-интерфейса для загрузки файла.

Автоматическое чтение и разбор (парсинг) строк файла.

Проверка (валидация) каждой записи на соответствие установленным правилам (наличие обязательных полей, корректность цен, остатков и гарантийных сроков).

Сохранение корректных данных в оперативную память (в коллекцию) для дальнейшего отображения в каталоге.

Информирование пользователя о результатах импорта: вывод статистики (сколько строк обработано, сколько успешно добавлено) и детального списка ошибок с указанием номеров строк, в которых они были допущены.

## 1.2. Требования к функциональности и ограничения реализации

Согласно индивидуальному заданию (Вариант 1. «Электроника»), к модулю предъявляются следующие функциональные требования:

1. Загрузка файла: Возможность выбора и загрузки CSV-файла через браузер (использование элемента input type="file").

2. Чтение данных: Построчное чтение файла с разделителями (запятая или точка с запятой).

3. Формирование объектов: Преобразование строковых значений CSV в объекты C# (модель Product).

4. Валидация данных: Проверка каждого поля на соответствие правилам:

   - Артикул (sku) должен быть уникальным и не пустым.
   - Цена (price) должна быть числом больше нуля.
   - Остаток на складе (stock) не может быть отрицательным.
   - Поле is_active должно иметь логическое значение (true/false).
   - Дополнительные поля для электроники: Бренд (brand) обязателен, Гарантия (warranty_months) целое число от 0 до 60.

5. Сохранение данных: Корректные товары должны добавляться в репозиторий в памяти (InMemoryProductRepository).

6. Вывод результата: Отображение таблицы с результатами импорта (Всего строк / Успешно импортировано / Количество ошибок) и списка ошибок с номерами строк.

7. Просмотр каталога: Отображение списка всех успешно загруженных товаров на отдельной странице.

К ограничениям реализации относятся следующие архитектурные и технические условия:

- Отсутствие базы данных: Это ключевое ограничение. Данные хранятся исключительно в оперативной памяти приложения (с

использованием коллекций List<T> или Dictionary). При перезапуске веб-сервера данные не сохраняются  это считается допустимым.

Отсутствие сложных парсеров: Для разбора CSV-файлов не требуется подключение сторонних библиотек (например, CsvHelper), достаточно стандартных средств работы со строками в C#.

Изоляция логики: Контроллер (Controller) не должен содержать код чтения файла и его валидации. Вся бизнес-логика должна быть инкапсулирована в отдельном сервисе (Service).

Отказоустойчивость: Приложение не должно аварийно завершать работу («падать») при загрузке некорректного файла, пустого файла или файла с неверными данными. Ошибки должны корректно обрабатываться и выводиться пользователю в понятном виде.

# 2. АРХИТЕКТУРА РЕШЕНИЯ

## 2.1. Общая структура проекта ASP.NET MVC

Приложение разработано с использованием фреймворка ASP.NET Core MVC и паттерна «Модель-Представление-Контроллер» (Model-View-Controller). Для соблюдения принципа разделения ответственности (Separation of Concerns) и повышения тестируемости кода, проект разделен на логические слои.

Процесс обработки данных при импорте можно представить в виде следующей схемы:

CSV-файл

↓

ProductImportService (Чтение, валидация и преобразование данных)

↓

InMemoryProductRepository (Хранение валидных объектов Product в памяти)

↓

ProductsController (Обработка HTTP-запросов и управление потоком данных)

↓

ASP.NET MVC Views (Отображение интерфейса пользователю)

Структура папок проекта организована следующим образом:

Models: содержит классы-сущности (Product, ImportResult, ImportError), описывающие структуру данных.

Repositories: содержит интерфейс и реализацию хранилища данных в памяти.

Services: содержит бизнес-логику по чтению файлов и валидации данных.

Controllers: класс ProductsController, который принимает запросы от браузера, вызывает сервисы и возвращает представления.

Views: HTML-шаблоны (Razor), отвечающие за визуализацию данных (страница импорта, список товаров, страница результатов).

### 2.2. Модель данных и репозиторий в памяти

Центральным элементом данных является модель Product. Поскольку предметной областью выбрана «Электроника», помимо базовых полей (артикул, название, цена, количество), в модель добавлены специализированные атрибуты: Brand (бренд), WarrantyMonths (срок гарантии в месяцах) и ModelNumber (номер модели).

Для хранения данных используется паттерн «Репозиторий». Создан интерфейс IProductRepository, который абстрагирует работу с хранилищем. Его реализация — класс InMemoryProductRepository — хранит товары в обычном списке List<Product>.

Использование интерфейса позволяет в будущем безболезненно заменить хранение в памяти на реальную базу данных (например, Entity Framework Core), не меняя код контроллера. Репозиторий предоставляет три основных метода:

Add(Product product) — добавление товара в коллекцию.

GetAll() — получение списка всех товаров.

GetBySku(string sku) — поиск товара по артикулу (используется для проверки на дубликаты).

### 2.3. Сервис импорта и валидация

Вся логика, связанная с обработкой файла, инкапсулирована в ProductImportService, который реализует интерфейс IProductImportService. Вынос этой логики из контроллера позволяет избежать его «раздувания» (так называемого Fat Controller) и делает код чистым и читаемым.

Сервис выполняет следующие функции:

1. Чтение потока данных: Принимает Stream файла и использует StreamReader для построчного чтения. Кодировка установлена как UTF-8 для корректного отображения кириллицы.

2. Парсинг: Разделяет строки на массив значений. Реализовано автоопределение разделителя (поддерживается как запятая ,, так и точка с запятой ;, которую часто использует Excel).

3. Валидация: Проверяет каждое значение на соответствие бизнес-правилам. Если данные некорректны (например, отрицательная цена), строка пропускается, а в специальный список ошибок (ImportError) добавляется запись с номером строки и описанием проблемы.

4. Формирование результата: Возвращает объект ImportResult, содержащий общее количество обработанных строк, количество успешно добавленных товаров и коллекцию найденных ошибок.

Для внедрения зависимостей (Dependency Injection) используется встроенный контейнер ASP.NET Core. В файле Program.cs репозиторий зарегистрирован как Singleton (чтобы данные хранились на протяжении всего времени работы приложения), а сервис импорта как Scoped.

# 3. ФОРМАТ CSV-ФАЙЛА И РЕАЛИЗАЦИЯ ФУНКЦИОНАЛЬНОСТИ

## 3.1. Описание формата CSV для предметной области «Электроника»

Для импорта данных используется текстовый файл формата CSV (Comma-Separated Values) с обязательной строкой заголовков. Поскольку в качестве предметной области выбран Вариант 1 («Электроника»), файл содержит как базовые поля товара, так и специализированные атрибуты.

Структура данных включает 12 колонок. Первая строка файла содержит названия полей (заголовки), разделенные запятой. Пример корректного CSV-файла:

sku,name,description,price,currency,category_name,stock,image_url,is_active,brand,warranty_months,model_number

1001,Ноутбук Lenovo IdeaPad 3,Бюджетный ноутбук для работы,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaPad3-15

1002,Мышь Logitech M185,Беспроводная мышь,1290,RUB,Периферия,10,/images/mouse.jpg,true,Logitech,12,M185

Описание полей и правила их валидации представлены в таблице 1.

Таблица 1 - Структура полей CSV-файла для электроники

| Поле | Описание | Правило валидации |
|---|---|---|
| sku | Артикул товара | Обязательно, уникальное значение |
| name | Название товара | Обязательно, не пустое |
| description | Описание | Может быть пустым |
| price | Цена | Обязательно, число > 0 |

| Поле | Описание | Правило валидации |
|---|---|---|
| currency | Валюта | По умолчанию RUB |
| category_name | Категория | По умолчанию «Без категории» |
| stock | Остаток на складе | Целое число, не меньше 0 |
| image_url | Ссылка на изображение | Может быть пустым |
| is_active | Активность товара | true или false |
| brand | Бренд (доп. поле) | Обязательно (для электроники) |
| warranty_months | Гарантия, мес. (доп. поле) | Целое число от 0 до 60 |
| model_number | Номер модели (доп. поле) | Может быть пустым |

### 3.2. Логика чтения и проверки данных

Обработка файла происходит в классе ProductImportService. Чтение реализовано построчно с использованием StreamReader в кодировке UTF-8, что гарантирует корректное отображение кириллицы.

Для защиты от ошибок пользователя, сервис автоматически определяет разделитель: если в строке присутствует точка с запятой (;, что часто бывает при сохранении файлов из Microsoft Excel в русской локали), разделитель меняется на нее, в противном случае используется стандартная запятая (,).

В процессе обработки каждой строки выполняется комплексная валидация данных. Если хотя бы одно правило нарушено, строка не добавляется в репозиторий, а в объект ImportResult записывается ошибка с указанием номера строки и причины.

Пример кода, демонстрирующий логику проверки данных в сервисе:

```
if (string.IsNullOrEmpty(sku)) {
    result.Errors.Add(new ImportError { LineNumber = lineNumber, Message = "Артикул (sku) обязателен" });
    continue;
}
if (_repository.GetBySku(sku) != null) {
    result.Errors.Add(new ImportError { LineNumber = lineNumber, Message = $"Артикул {sku} уже существует" });
    continue;
}
if (!decimal.TryParse(priceStr, out var price) || price <= 0) {
    result.Errors.Add(new ImportError { LineNumber = lineNumber, Message = "Цена должна быть числом больше 0" });
    continue;
}
if (!int.TryParse(warrantyStr, out var warranty) || warranty < 0 || warranty > 60) {
    result.Errors.Add(new ImportError { LineNumber = lineNumber, Message = "Гарантия должна быть от 0 до 60 месяцев" });
    continue;
}
```

Если все проверки пройдены успешно, создается объект Product, который передается в репозиторий методом Add(). Счетчик успешно импортированных записей увеличивается.

### 3.3. Разработка интерфейса

Для взаимодействия пользователя с модулем импорта в приложении реализованы контроллер ProductsController и три представления (Views).

Контроллер ProductsController обрабатывает HTTP-запросы:

Index() (GET) формирует страницу со списком всех товаров, получая их из репозитория.

Import() (GET) возвращает представление с формой загрузки файла.

Import(IFormFile file) (POST) принимает файл из формы, проверяет его наличие, передает поток данных в ProductImportService и возвращает представление с результатами (ImportResult).

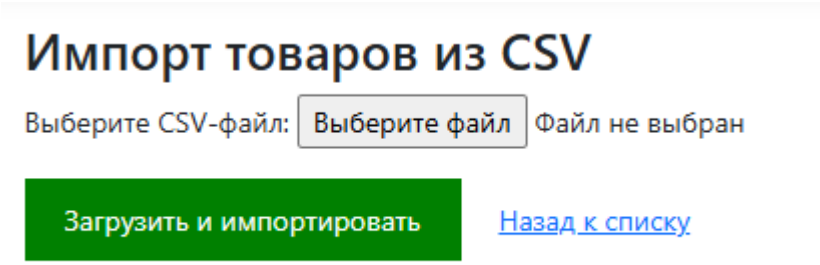



Рисунок 1 импорт товаров

Представления (Views) разработаны с использованием Razor-синтаксиса:

1. Import.cshtml содержит HTML-форму с атрибутом enctype="multipart/form-data", что необходимо для передачи файлов. Форма включает поле input type="file" и кнопку отправки.

2. ImportResult.cshtml отображает статистику импорта (всего строк, успешно, ошибок) и, при наличии ошибок, выводит их в виде таблицы с указанием номера строки и текста ошибки.

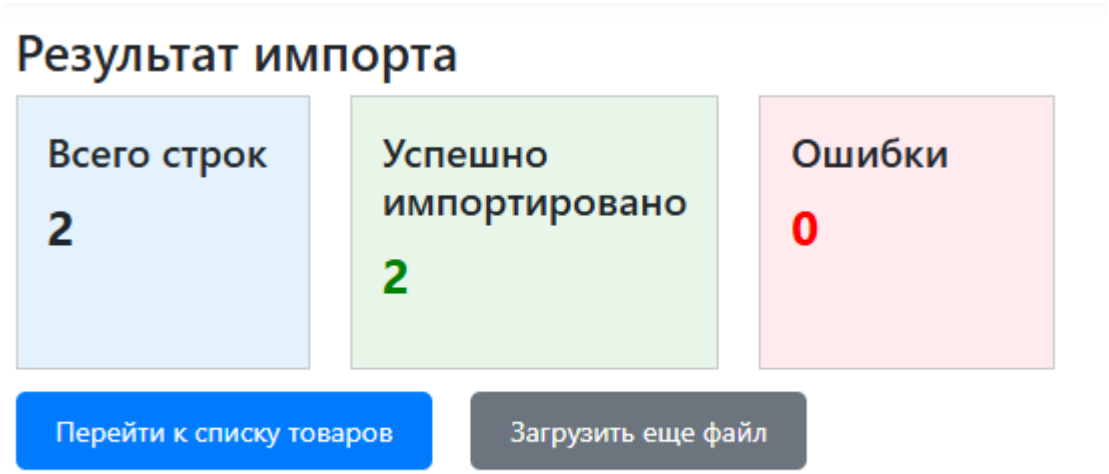



Рисунок 2 результат импорта

3. Index.cshtml выводит каталог товаров в виде таблицы. В шапке таблицы отображаются наименования колонок (Артикул, Название, Бренд, Цена, Категория, Остаток, Гарантия, Статус).

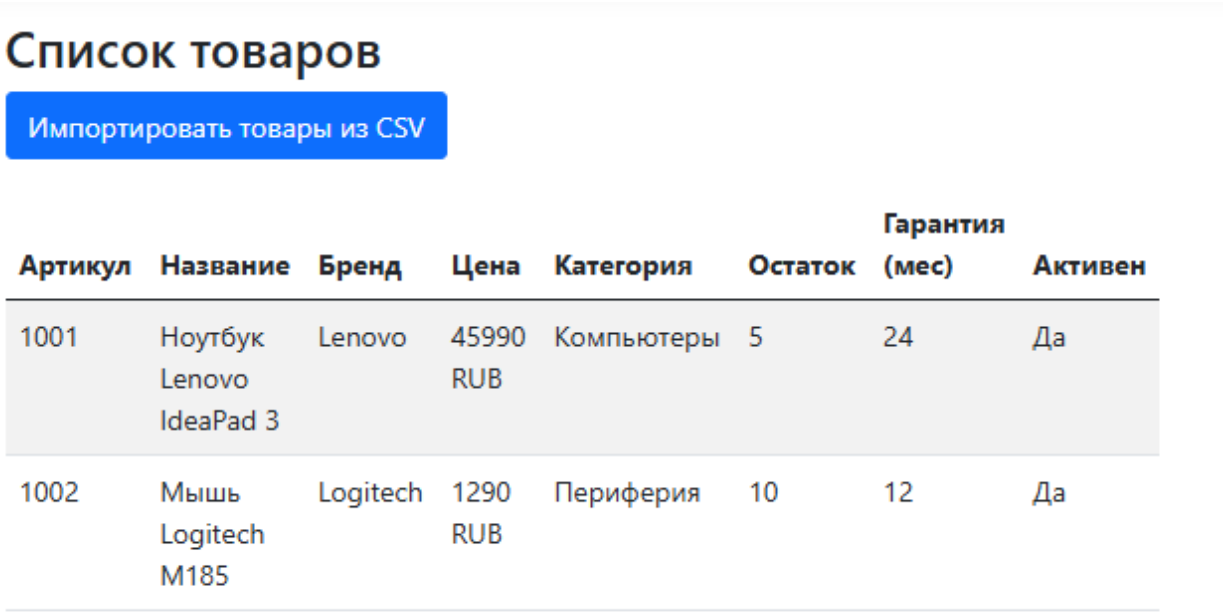



Рисунок 3 список товаров

Интерфейс адаптирован для просмотра в браузере, использует базовую CSS-стилизацию (цветовое выделение карточек статистики и ошибок) для удобства восприятия информации сотрудником магазина.

# 4. ТЕСТИРОВАНИЕ И ОТЛАДКА

## 4.1. Разработанные тестовые сценарии

Согласно требованиям к программному модулю, было разработано шесть тестовых сценариев. Тестирование проводилось вручную с использованием браузера (Google Chrome) и заранее подготовленных CSV-файлов. Основная цель - проверить, что приложение корректно обрабатывает как валидные данные, так и ошибочные, не завершаясь аварийно.

В таблице 2 представлены разработанные тестовые сценарии.

Таблица 2 - Тестовые сценарии для модуля импорта

| № | Сценарий | Входные данные | Ожидаемый результат |
|---|---|---|---|
| 1 | Успешный импорт корректного файла | valid_products.csv (2 товара с заполненными полями для электроники) | 2 товара добавлены в память, ошибок нет. Товары отображаются на странице списка. |
| 2 | Файл с неверной ценой | invalid_price.csv (цена = -500) | Товар не добавлен. Ошибка по строке 2: «Цена должна быть больше нуля». |
| 3 | Пустой файл | empty_file.csv (файл без данных) | Приложение не падает. Импортировано 0 товаров. Сообщение: «Файл пуст». |
| 4 | Дубликат артикула | duplicate_sku.csv (два товара с одинаковым SKU) | Первый товар добавлен, второй пропущен. Ошибка: «Артикул уже существует». |

| № | Сценарий | Входные данные | Ожидаемый результат |
|---|---|---|---|
| 5 | Отрицательный остаток | negative_stock.csv (остаток = -5) | Товар не добавлен. Ошибка: «Остаток не может быть отрицательным». |
| 6 | Файл с кириллицей (Windows-1251 / UTF-8) | cyrillic_test.csv (русские названия) | Названия товаров отображаются корректно, без «кракозябр» в браузере. |

### 4.2. Результаты функционального тестирования

В процессе тестирования все сценарии были выполнены. Приложение успешно справилось с обработкой некорректных данных, не допустив необработанных исключений (крашей).

Сценарий 1 (Успешный импорт). При загрузке файла с двумя товарами (например, ноутбук и мышь) система вывела статистику: Всего строк: 2, Успешно импортировано: 2, Ошибки: 0. При переходе на страницу списка товары корректно отобразились в таблице.




Рисунок 4 результат импорта с двумя товарами

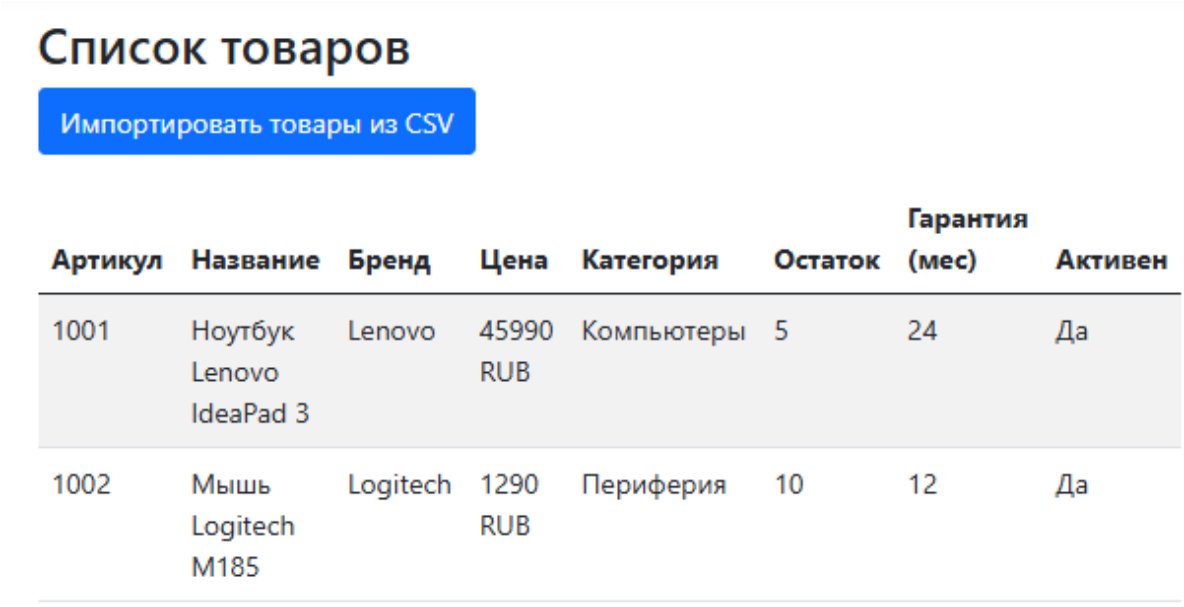



Рисунок 5 результат загрузки товаров

Сценарий 2 и 5 (Ошибки валидации). При загрузке файлов с отрицательной ценой и отрицательным остатком система не упала, а вывела страницу с результатами, где в красной карточке «Ошибки» было указано их количество, а в таблице ниже - номер строки (2) и точный текст ошибки («Цена должна быть числом больше 0» и «Остаток не может быть отрицательным» соответственно).

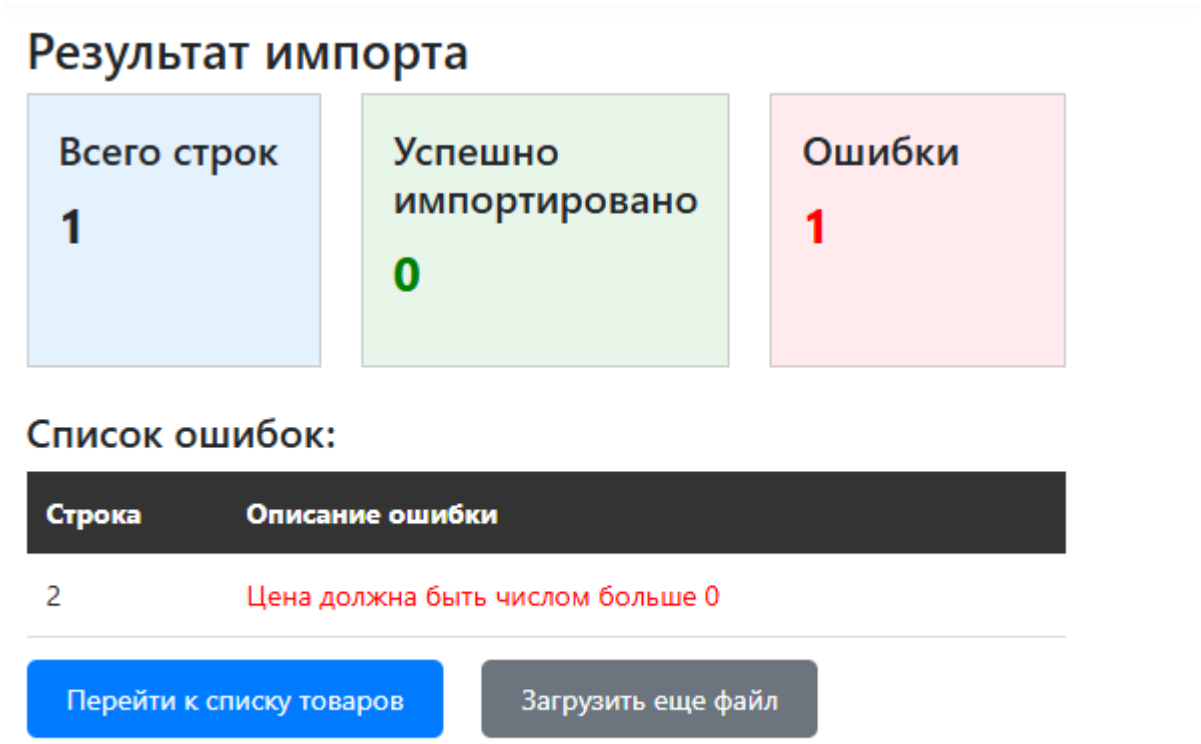



Рисунок 6 ошибки валидации

Сценарий 4 (Дубликат артикула). При загрузке файла, где вторая строка содержала уже существующий SKU, первый товар успешно добавился, а для второго сработала проверка GetBySku, и пользователю была выведена ошибка «Артикул 1001 уже существует».

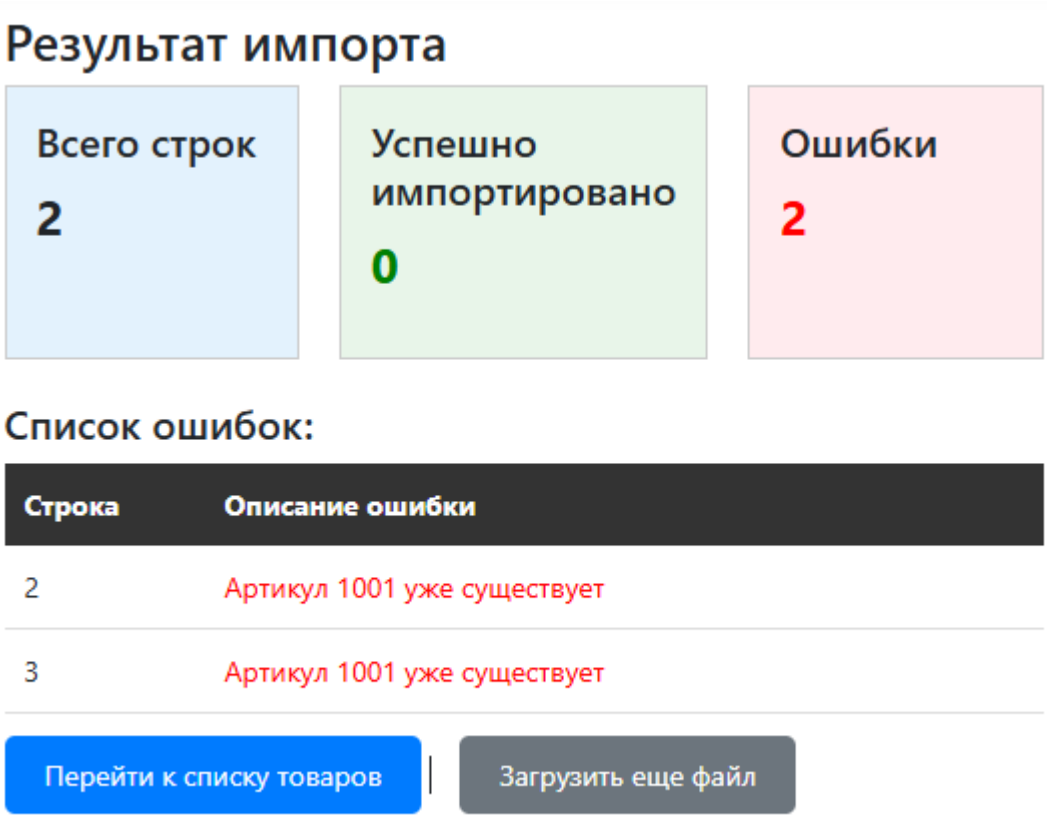



Рисунок 7 дубликат артикула

Сценарий 6 (Кириллица). Файл был сохранен в кодировке UTF-8. Благодаря использованию StreamReader с явным указанием кодировки, русские названия товаров («Ноутбук», «Мышь») корректно отобразились на странице списка.

Отладка программного модуля (ПК 2.3).

# ЗАКЛЮЧЕНИЕ

За время прохождения производственной практики в Приемной комиссии Пермского института (филиала) РЭУ им. Г.В. Плеханова была выполнена комплексная работа по разработке программного модуля импорта товаров из CSV-файлов для электронного магазина (предметная область - «Электроника»).

В ходе выполнения индивидуального задания были решены следующие задачи:

1. Проведен анализ технического задания и сформированы требования к программному модулю, включая правила валидации данных для электроники (проверка бренда, срока гарантии и номера модели).

2. Спроектирована архитектура веб-приложения на базе ASP.NET Core MVC с соблюдением принципов разделения ответственности. Логика работы с файлами и валидации вынесена в сервисный слой, а хранение данных реализовано через паттерн «Репозиторий».

3. Разработан веб-интерфейс (контроллер и представления), позволяющий пользователю загружать CSV-файлы, просматривать статистику успешного импорта, список возникших ошибок с указанием номеров строк, а также каталог загруженных товаров.

4. Проведено функциональное тестирование по шести сценариям, включая обработку пустых файлов, дубликатов артикулов и неверных числовых значений. Приложение продемонстрировало отказоустойчивость и корректную обработку исключительных ситуаций.

Возникшие трудности и способы их решения:

В процессе разработки возник ряд технических сложностей, которые были успешно преодолены:

Проблема с кодировкой кириллицы и разделителями. Файлы, сохраняемые из Microsoft Excel, могли использовать точку с запятой (;) в качестве разделителя и иметь отличную от UTF-8 кодировку. Решение: в классе ProductImportService было реализовано автоопределение разделителя, а StreamReader настроен на работу с кодировкой UTF-8 с обнаружением BOM.

Конфликт отладчика Visual Studio при загрузке файлов. При выборе файла через диалоговое окно встроенный отладчик аварийно завершал процесс из-за конфликта функции «Горячая перезагрузка» (Hot Reload). Решение: был проведен анализ логов через окно «Вывод», после чего запуск приложения осуществлялся в режиме «Запуск без отладки» (Ctrl+F5), а функция Hot Reload была отключена в настройках среды.

В результате прохождения практики были успешно освоены следующие профессиональные компетенции:

ПК 2.1 - разработка требований к программным модулям на основе анализа технической документации (описание полей CSV и бизнес-правил).

ПК 2.2 - выполнение интеграции модулей в программное обеспечение (внедрение сервиса импорта и репозитория в ASP.NET MVC-приложение).

ПК 2.3 - выполнение отладки программного модуля с использованием специализированных средств (работа с отладчиком Visual Studio, обработка исключений).

ПК 2.4 - разработка тестовых наборов и сценариев для программного обеспечения (создание 6 тестовых сценариев и их проверка).

ПК 2.5 - инспектирование компонент ПО на предмет соответствия стандартам кодирования (использование осмысленных имен переменных, разделение логики по слоям, обработка ошибок).

Направления для дальнейшего развития:

Разработанный модуль имеет потенциал для улучшения. В перспективе можно добавить:

Хранение данных в реальной базе данных (например, с использованием Entity Framework Core и PostgreSQL), чтобы данные не терялись при перезапуске сервера.

Асинхронный импорт больших файлов (использование async/await) для предотвращения блокировки интерфейса.

Расширение каталога товаров: добавление пагинации (постраничного вывода), поиска по названию и фильтрации по бренду или категории.

Написание unit-тестов (например, с использованием фреймворка xUnit) для автоматической проверки логики валидации.

Таким образом, цель производственной практики достигнута в полном объеме, полученные навыки веб-разработки на языке C# закреплены на практике, а разработанный модуль готов к использованию.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ


1. Албахари, Дж. C# 10 и .NET 6. Современная кроссплатформенная разработка / Дж. Албахари, Б. Албахари. - 6-е изд. - Москва : ДМК Пресс, 2022. - 1008 с.

2. Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. - Санкт-Петербург : Питер, 2016. - 368 с.

3. Макконнелл, С. Совершенный код. Практическое руководство по разработке программного обеспечения / С. Макконнелл. - 2-е изд. - Москва : Русская редакция, 2010. - 896 с.

4. Троелсен, Э. Язык программирования C# 7 и платформы .NET и .NET Core / Э. Троелсен, Ф. Джепикс. - 8-е изд. - Москва : Вильямс, 2018. - 1360 с.

5. Фаулер, М. Архитектура корпоративных программных приложений / М. Фаулер. - Москва : Вильямс, 2006. - 544 с.

6. Фримен, А. ASP.NET Core MVC с примерами на C# для профессионалов / А. Фримен. - 8-е изд. - Москва : Диалектика, 2022. - 1008 с.

7. Документация по ASP.NET Core [Электронный ресурс] // Microsoft Learn. - Режим доступа: https://learn.microsoft.com/ru-ru/aspnet/core/ (дата обращения: 10.11.2026).

8. Документация по языку C# [Электронный ресурс] // Microsoft Learn. - Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/csharp/ (дата обращения: 10.11.2026).

9. Паттерн Repository (Репозиторий) [Электронный ресурс] // Refactoring.Guru. - Режим доступа: https://refactoring.guru/ru/design-patterns/repository (дата обращения: 10.11.2026).

10. Работа с файлами и потоками в C# [Электронный ресурс] // METANIT.COM. - Режим доступа: https://metanit.com/sharp/tutorial/5.1.php (дата обращения: 10.11.2026).

11. Руководство по Git [Электронный ресурс] // Git SCM. - Режим доступа: https://git-scm.com/book/ru/v2 (дата обращения: 10.11.2026).